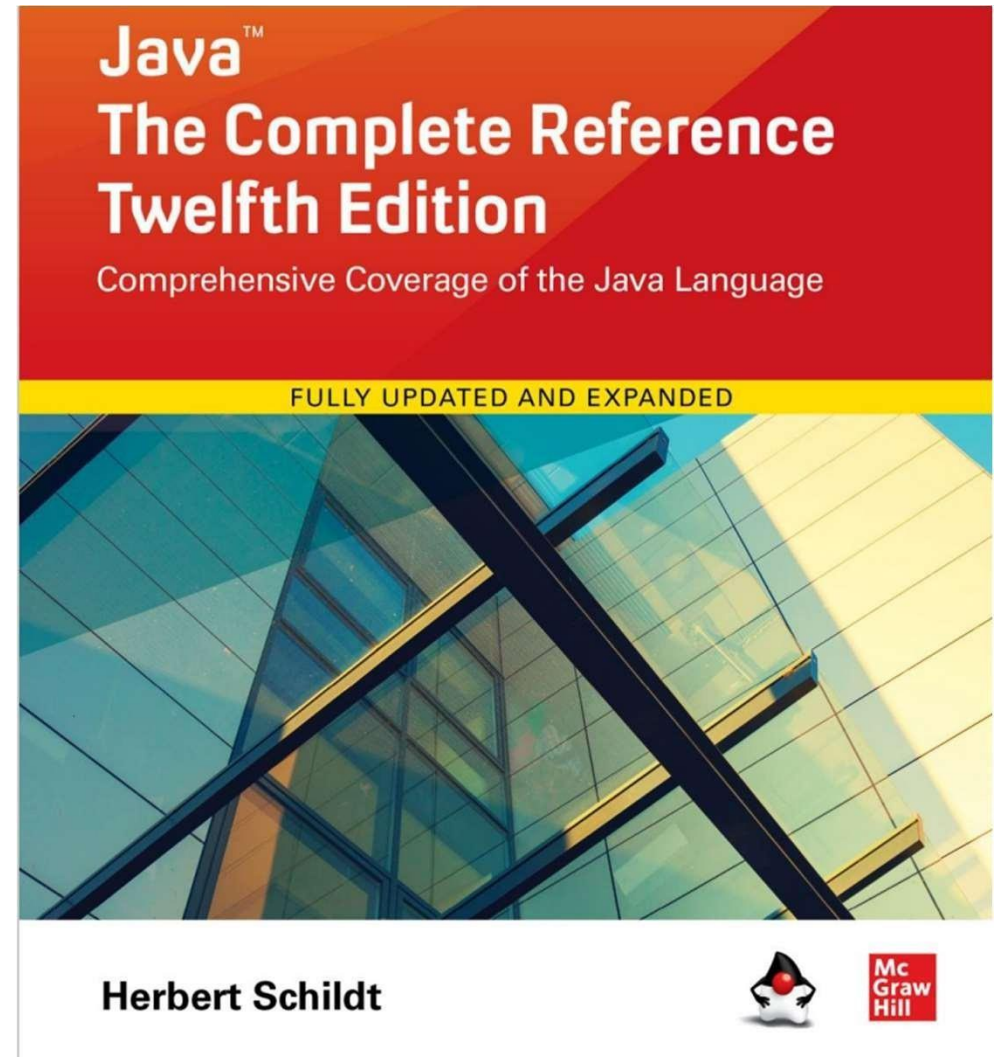


JAVA-Module-1

Module-1

1.1	Introduction to Java and OOP Principles ,JDK, JVM, Bytecode, simple java program
1.2	Data Types, Variables, and Arrays: Type Conversion and Casting, Automatic Type Promotion in Expressions, Arrays, Operators
1.3	Control Statements: Java's Selection Statements (if, The Traditional switch), Iteration Statements (while, do- while, for, The For-Each Version of the for Loop), Jump Statements (Using break, Using continue, return).

Module-1



Java's Lineage

- Java is related to C++, which is a direct descendant of C.
- **From C, Java derives its syntax.** Many of Java's object-oriented features were influenced by C++.
- While it is true that Java was influenced by C++, **it is NOT an enhanced version of C++.**

Java Buzz Words

- The Java programming language is a high-level language that can be characterized by all of the following buzzwords.
 - Simple
 - Object oriented
 - Distributed
 - Multithreaded
 - Dynamic
 - Architecture neutral
 - Portable
 - High performance
 - Robust
 - Secure

[The Java Language Environment \(oracle.com\)](http://oracle.com)

Java Buzz Words- **Simple**

- Java is designed as a simple programming language.
- Java is simple and follows C/C++ syntax.
- When Java was developed, the Java team wanted it to be simple for the professional programmer to learn and use effectively.
- **Simpler than C++... Why?**
 - No pointers and hence no pointer maths
 - No structs, typedefs
 - Automatic garbage collection
 - No operator overloading
 - No templates
 - No multiple inheritance

Java Buzz Words- **Portable**

- Java programs can be executed on any kind of computer containing JVM
- When java program is compiled it generates a intermediate code file called “Bytecode”.
- Bytecode helps java to achieve portability.
- This bytecode is taken to any computer with any OS and executed directly.

Java Buzz Words- **Object Oriented**

- Java follows purely object oriented model. This means java program is developed by using classes and objects.
- Problems and their solutions are packaged in terms of classes.
 - The information in a class is the data
 - The functionality in a class is the method
- An object is anything that exists in the real world. For example, every human being, a book, a tree, pen, pencil, and so on.
- Every object has properties and shows certain behavior (action). Let us understand this statement with the help of an example. Consider a dog. The properties of a dog are name, height, color, age, etc. These properties are represented by variables in java. The object dog has several behaviors (actions) such as running, barking, eating, sleeping, etc.
- Java supports the following OO features
 - Encapsulation
 - Inheritance
 - Polymorphism
 - Abstraction

Java Buzz Words- **Robust**

- The meaning of Robust is “strong”. This means that Java programs are strong because they do not crash easily like C or C++ programs.
- Strongly typed language
- No Pointers
- Automatic Garbage Collection. Java manages all dynamic memory and hence has robust memory management.
- Extensive error checking and Exception handling

Java Buzz Words- **Architecture Neutral**

- Bytecode helps java to achieve portability
- Java Bytecode can be executed on computers having any kind of OS and any kind of CPU.
- Since java applications can run on any kind of CPU , Java is architecture neutral

Java Buzz Words- **Interpreted**

- The compiled code of java is not machine instructions but rather it's a intermediate code called "Bytecode".
- This Bytecode can be executed on any machine that implements JVM (Java Virtual Machine)
- JVM interprets the bytecode into machine instructions at run time.

Java Buzz Words- **Distributed**

- Designed for the internet/web.
- Build in support of TCP-IP protocols, HTTP/FTP protocols
- RMI – Remote method Invocation

Java Buzz Words- **Secure**

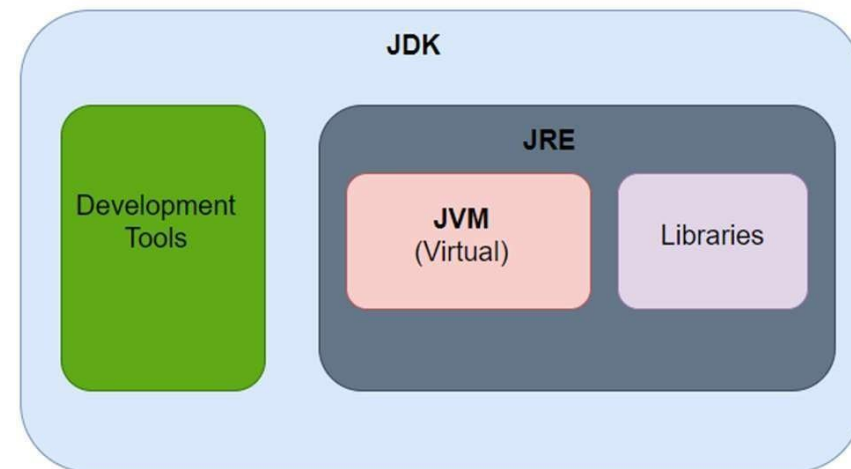
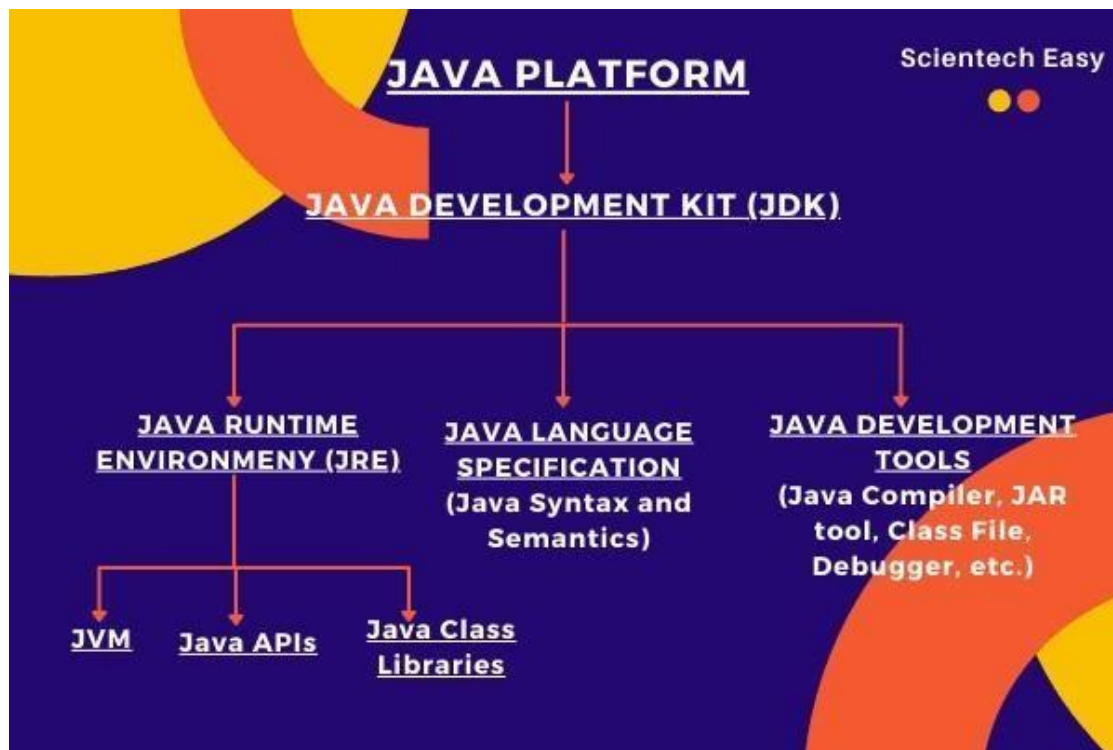
- Interpreted.
 - Java interpreter applies several tests to the compiled code to check for illegal code. Performs no illegal data conversions, performs only legal object field accesses and all opcode parameter types are checked for legality.
- Bytecode checker and verifier
- JVM
- No pointers
 - Features of C/C++ that often results in illegal memory accesses are not in Java language.

Java Buzz Words- **Multithreaded**

- Interactive programs
- Build in support for threads
- Build in support for process synchronization

Java Architecture

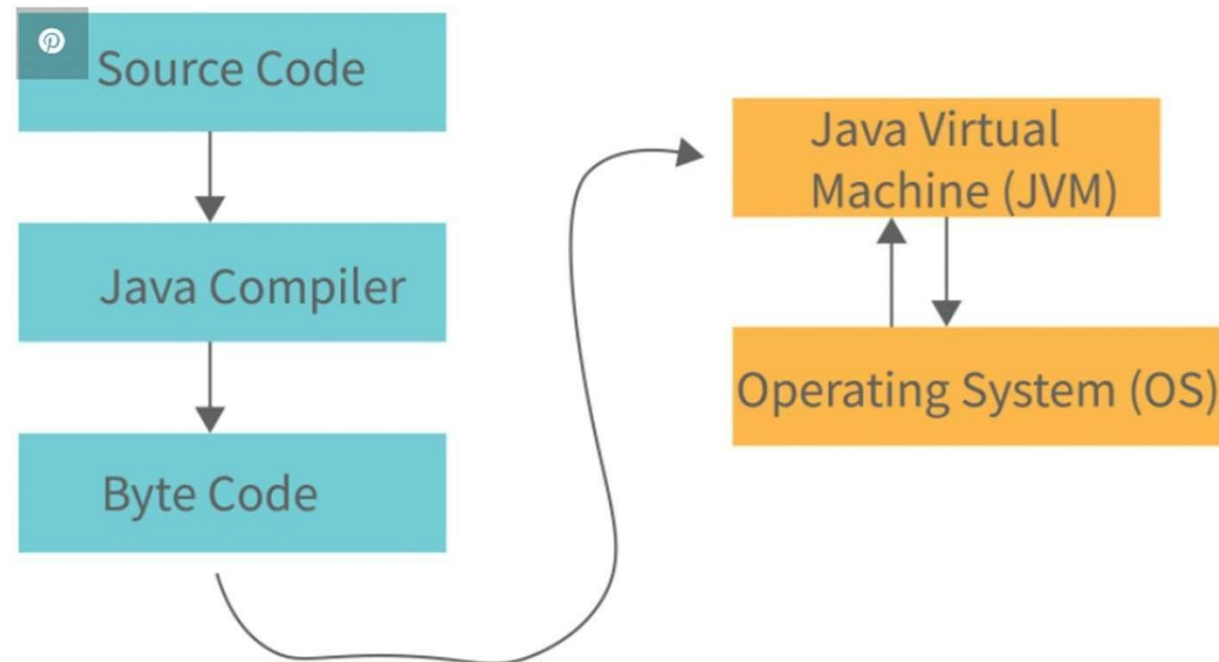
- Java-Architecture is composed of a **hierarchy of components** that is shown in the below figure.



Java Architecture

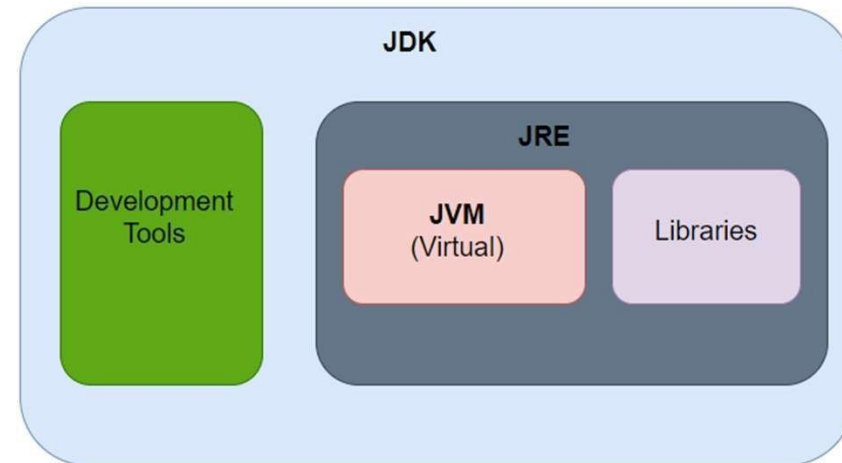
- The JVM, JRE, and JDK are all components of Java Architecture.
- It combines the interpretation and compilation processes.
- It enumerates all of the steps involved in writing a Java program.
- The below-mentioned points and diagram will simply illustrate Java architecture.

1. There is a compilation and interpretation process in Java.
2. After the JAVA code is written, the JAVA compiler comes into the picture that converts this code into byte code that can be understood by the machine.
3. After the creation of bytecode JAVA virtual machine(JVM) converts it to the machine code, i.e. (.class file)
4. And finally, the machine runs that machine code.



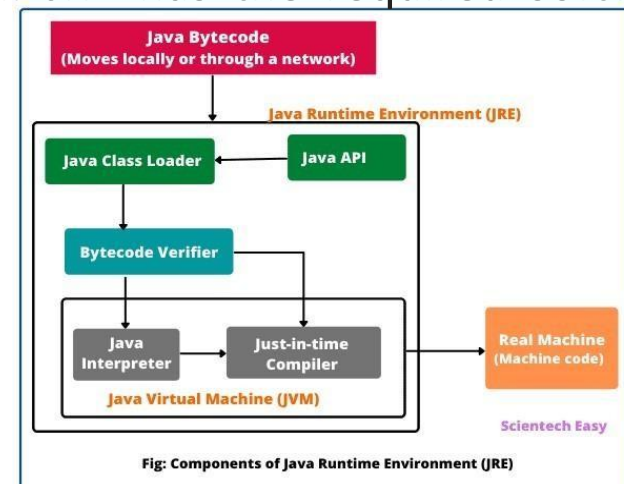
Java Development Kit (JDK)

- JDK is a software development environment that is developed and distributed by Oracle.
- JDK is used for building(developing) Java software applications and applets.
- JDK is a bundle of software development tools and supporting libraries combined with JRE (Java Runtime Environment).
- We cannot compile java programs without JDK installed.
- The development tools are
 - Javac (java compiler)
 - Java (Java launcher)
 - Javadoc (Documentation tool)
 - Appletviewer (can be used to run and debug java applets without a web browser)



Java Runtime Environment(JRE)

- Java Runtime Environment (JRE) is a set of software tools that **provides the runtime environment necessary for executing Java applications.**
- It includes a virtual machine (JVM) and class libraries, enabling Java programs to run on different operating systems.
- Java Runtime Environment provides an platform where all the applications like JVM and other runtime libraries linked together to run your Java Program.
- It builds a runtime environment where you can execute the Java program.
- The JRE also initiates the JVM for its execution. JRE has the required software and libraries to run the programs.
- JRE consists of the following main components.
 - Java API (Application Programming Interface)
 - Class Loader
 - Bytecode verifier
 - JVM interpreter



Java Runtime Environment(JRE)

- **Why is the JRE important?**

- The JRE is crucial because it **bridges the gap between Java applications and the underlying operating system**. It allows Java programs to be executed on various platforms, following the “write once, run anywhere” principle.

- **How does the JRE work?**

- The JRE works by taking **Java bytecode**, which is generated from human-readable Java code, and translating it into machine code, making it executable on the host system.

- **What are the components of the JRE?**

- Its components, including the Java Virtual Machine and class libraries, contribute to the “write once, run anywhere” principle, making Java a versatile and **widely-used programming language**. Understanding the Java environment is essential for developers to create efficient, compatible, and secure Java applications.

What is Java API (Application Interface Programming)?

- Java Application Programming Interface (API) is a very large collection of prepackaged, ready-made software components that provides the core functionality of the Java programming language.
- In simple words, Java API is a large collection of already defined classes, interfaces, and methods in the form of Java packages.
- It provides many useful capabilities, such as Graphical User Interface (GUI), Date, Time, and Calendar capabilities to programmers.
- Java API is grouped into libraries of related classes and interfaces along with their fields, constructors, and methods. These libraries are provided in the form of packages.
- Some popular libraries and their functionality from the Java API, in short, are as follows:
 - **Java.lang:** It is a package that provides fundamental classes to design the Java programming language. The Java.lang package is dynamically imported (i.e. loaded) in a Java program. It does not need to import explicitly.
 - The most commonly used classes from Java.lang package are Double, Float, Integer, String, StringBuffer, System, and Math.
 - **Java.io:** It supports Input/Output through the file system, keyboard, network, etc. The java.io package contains several classes to perform input and output operations.
 - The most commonly used classes from java.io are File class, InputStreams, OutputStreams, Readers, Writers, and RandomAccessFile.

What is Java API (Application Interface Programming)?

- **Java.util:** It supports various programming utilities. The java.util package provides legacy collection classes, event model, collections framework, date and time capabilities, and other utility classes such as string tokenizer.
 - **Java.math:** It is used to support mathematical operations.
 - **Java.security:** It supports security functions.
 - **Java.awt:** It supports creating graphical user interface (GUI), painting graphics, and images.
 - **Java.sql:** It support for accessing relational databases through SQL.
 - **Java.net:** The java.net package provides classes that are used for implementing networking in java programs.
 - **Java.imageIO:** It support for image input/output.
-
- The JRE's ByteCode verifier makes sure that the code that is passed to the interpreter is correct, passed all the verifier tests, and the file could execute properly.

Differences Between Java Runtime Environment (JRE) and Java Development Kit (JDK)

Many people confuse the Java Runtime Environment (JRE) with the Java Development Kit (JDK), but they serve different purposes in the **Java development process**.

JRE is primarily responsible for executing Java applications and applets. It provides the necessary runtime environment, including JVM and class libraries, but it does not include development tools such as compilers and debuggers.

On the other hand, **JDK is a comprehensive software development kit** that includes everything provided in JRE and additional development tools like the Java Compiler (javac), debugger (jdb), and other utilities.

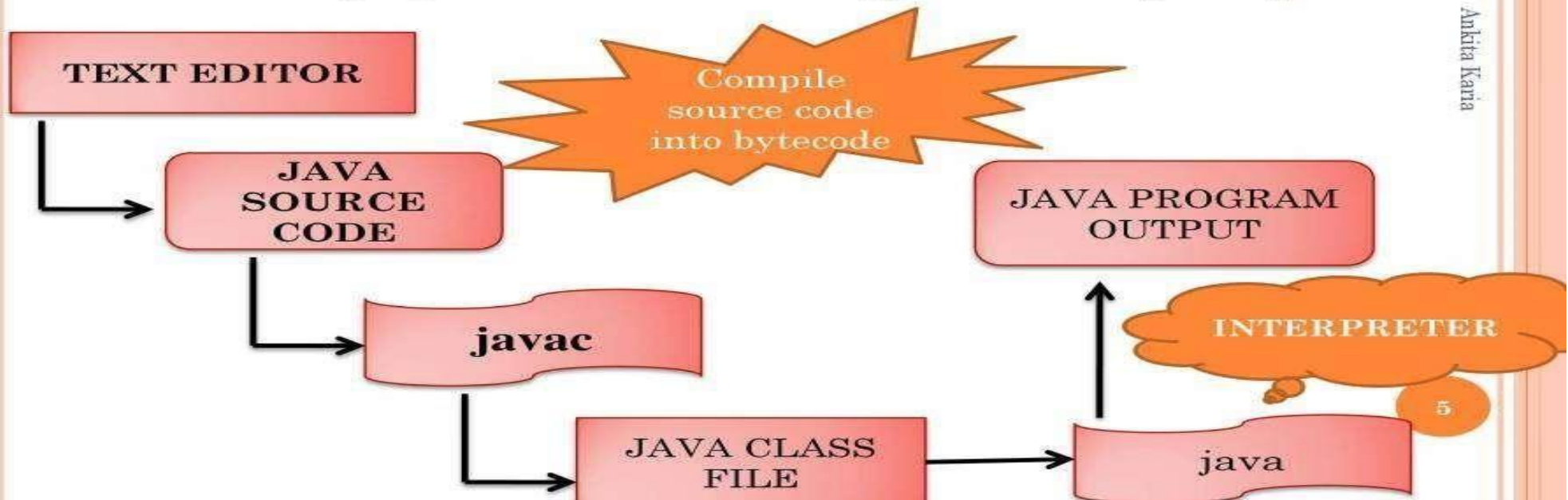
JDK is essential for developers who want to create, compile, and debug Java applications.

In essence, **JRE is for running Java programs, while JDK is for developing and running them.**

Execution Steps in Java

EXECUTION STEPS IN JAVA

- To create a program in JAVA, we need to create a source code file using a text editor.
- The source code is then compiled using the JAVA COMPLIER *javac*
- And then the program is executed using JAVA interpreter *java*.



Comparison of C and Java HelloWorld Program

C++ program

```
void main()
{
    cout<< " My first
    C++ program";
    getch();
}
```

JAVA Program

```
class First
{
    public static void main( String args[])
    {
        System.out.println("My first
        JAVA program");
    }
}
```

Since, JAVA is a true OO Language,
Everything must be placed inside a class.

Java main entry function explanation

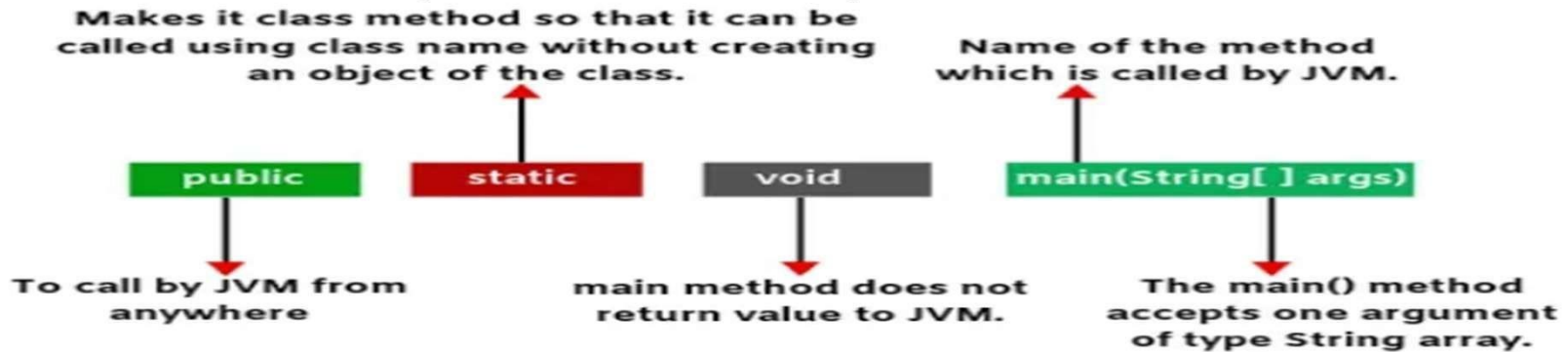
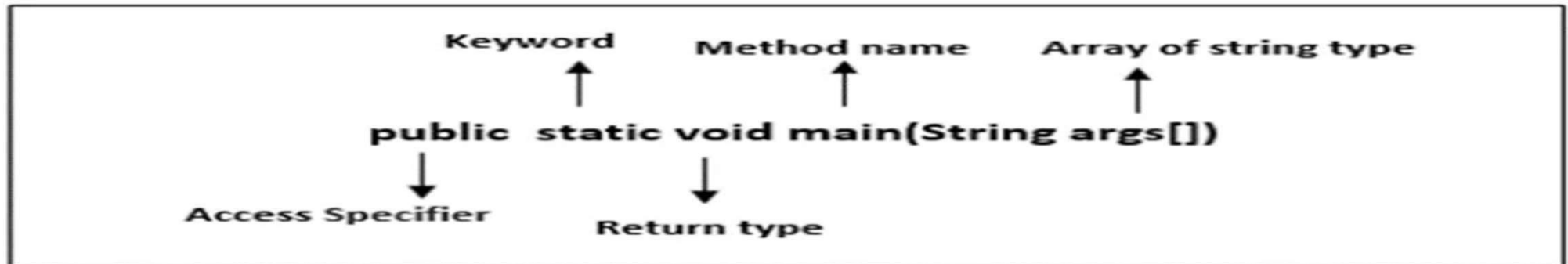


Fig: Java main method

`public static void main (String args [])` (Source: scientecheasy)



<https://medium.com/@sunkavallisowjanya/what-does-public-static-void-main-string-args-df5790cb037c>

<https://www.geeksforgeeks.org/java-main-method-public-static-void-main-string-args/>

Compile and run a Java program

Step 1. Write a simple Java program

Create a new text file, called `Intro.java` and place it somewhere on your computer, like in your documents folder.

Next, add the code from Listing 1, which is a very simple Java program.

Listing 1. Intro.java

```
public class Intro {  
  
    public static void main(String[] args) {  
        System.out.println("Welcome to the JDK!");  
    }  
  
}
```

Compile and run a Java program

Step 2. Compile with the JDK

Next, use the JDK compiler to turn your text file into an executable program. Compiled code in Java is known as bytecode, and carries the `.class` extension.

You'll use the `javac` command, passing the `Intro.java` file as the argument to the command. You are feeding the Java file we created in Listing 1 into the `javac` command. On my system, that looks like Listing 2. (In older Java versions, you'll need to type the full path to the command into your command shell as in the commented line.)

Listing 2. Compile with the JDK

```
javac Intro.java  
// "C:\Program Files\Java\jdk-10.0.1\bin\javac.exe" Intro.java
```

That should result in a successful compile. The `javac` will not respond with a success message; it will just output the new file. Any errors will result in console output.

Compile and run a Java program

Step 3. Run the .class file

You should now see the `Intro.class` file in the same directory as `Intro.java`.

You can run it by typing: `java Intro`, which will result in the output shown in Listing 3. Note that you don't include the `.class` when typing this command.

Listing 3. Running Intro.class

```
C:\Users\mtyson\Documents>java Intro
Welcome to the JDK!
```

Simple Java Programs - Addition of Two Numbers- Hardcoded Input

```
public class AddTwoNumbersWithoutScanner {  
    public static void main(String[] args) {  
        // Provide two numbers directly  
        double num1 = 5.5;  
        double num2 = 10.5;  
  
        // Calculate the sum of the two numbers  
        double sum = num1 + num2;  
  
        // Display the result (sum)  
        System.out.println("The sum of " + num1 + " and " + num2 + " is: " + sum);  
    }  
}
```


Simple Java Programs - Addition of Two Numbers- Using Scanner class

```
1  import java.util.Scanner;
2
3  public class AddNumbers_Scanner {
4      public static void main(String[] args) {
5          // Create a Scanner object to read input from the user
6          Scanner scanner = new Scanner(System.in);
7
8          // Prompt the user to enter the first number
9          System.out.print("Enter the first number: ");
10         double num1 = scanner.nextDouble();
11
12         // Prompt the user to enter the second number
13         System.out.print("Enter the second number: ");
14         double num2 = scanner.nextDouble();
15
16         // Close the Scanner to avoid resource leak
17         scanner.close();
18
19         // Calculate the sum of the two numbers
20         double sum = num1 + num2;
21
22         // Display the result (sum)
23         System.out.println("The sum of " + num1 + " and " + num2 + " is: " + sum);
24     }
25 }
26
```

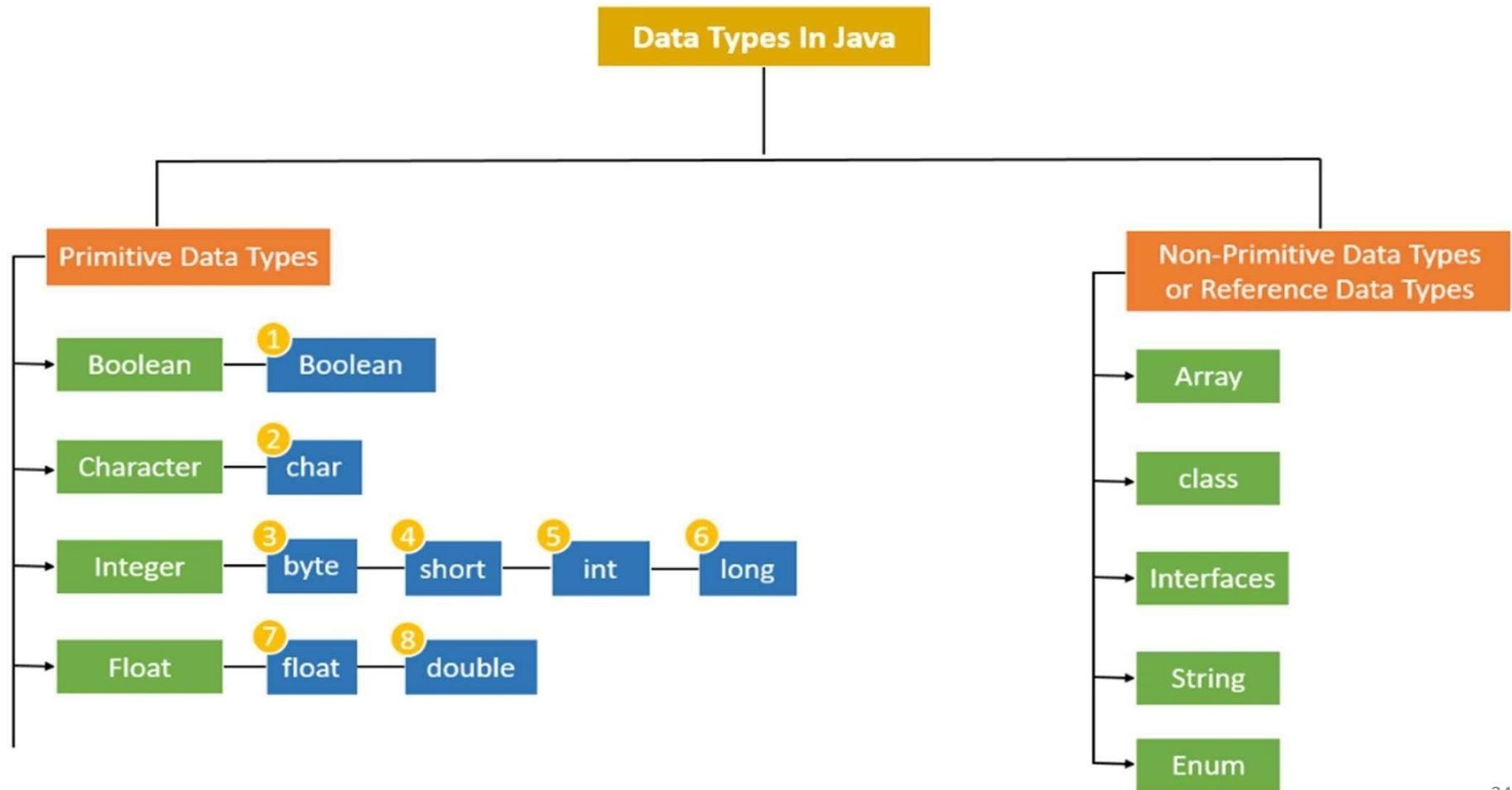
Simple Java Programs - Addition of Two Numbers- Using Command Line Arguments

```
1  public class AddNumbers {
2      public static void main(String[] args) {
3          if (args.length != 2) {
4              System.out.println("Usage: java AddNumbers <num1> <num2>");
5              return;
6          }
7
8          // Parse command-line arguments
9          double num1 = Double.parseDouble(args[0]);
10         double num2 = Double.parseDouble(args[1]);
11
12         // Perform addition
13         double sum = num1 + num2;
14
15         // Display the result
16         System.out.println("Sum: " + sum);
17     }
18 }
```


Data-Types

- One of the fundamental concepts in computer programming is memory, used to store information in the computer.
- Computer use bits as the smallest information that can be stored. A bit is either 0 or 1
- Group of 8-bits is called as a byte.
- We need a way to store different kinds of data in the computer's memory and tell the computer what kind of data is stored at what memory location.
- Data-Types are the way for us to specify what kind of data and the size we need to store at a given memory location.
- Examples for a data-type are integer, float, character etc.
- Data-Types in Java are divided into two categories.
 - Primitive
 - Non-Primitive Or Reference Data-Types

Data-Types



Data-Types (Size and Range)

Primitive Data Types Table – Default Value, Size, and Range

Data Type	Default Value	Default size	Range
byte	0	1 byte or 8 bits	-128 to 127
short	0	2 bytes or 16 bits	-32,768 to 32,767
int	0	4 bytes or 32 bits	2,147,483,648 to 2,147,483,647
long	0	8 bytes or 64 bits	9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	0.0f	4 bytes or 32 bits	1.4e-045 to 3.4e+038
double	0.0d	8 bytes or 64 bits	4.9e-324 to 1.8e+308
char	'\u0000'	2 bytes or 16 bits	0 to 65536
boolean	FALSE	1 byte or 2 bytes	0 or 1

Primitive Data-Types (Boolean)

- A boolean data type can store either True or False.
- They can be used to check whether two values are equal or not (basically in conditional statements to return True or False). Typically, programmers use it as a flag variable to track true or false conditions.
- The default boolean value is False.

```
class BooleanDataTypes
{
    public static void main(String args[]) {
        boolean var1 = true;
        if (var1 == true) //checks if the value is true or false
        {
            System.out.println("Boolean value is True");
        }
        else
        {
            System.out.println("Boolean value is False");
        }
    }
}
```

Output-

```
Boolean value is True
```

Primitive Data-Types (Character)

- The char data type stores a single character.
- It stores lowercase and uppercase characters, which must be enclosed in single quotes.
- The char data type in Java supports Unicode characters and provides provision to multiple languages like English, French, German, etc.
- It takes memory space of 16 bits or 2 bytes. The values stored range between 0 to 65536.

```
class CharDataType {  
    public static void main(String[] args) {  
        char var1 = 'A';  
        char var2 = 'd';  
        System.out.println(var1);  
        System.out.println(var2);  
    }  
}
```

Output:

```
A  
d
```

Primitive Data-Types (Integer)

Integer type –

An integer type stores an integer number with no fractional or decimal places. Java has four integer types – byte, short, int, and long.

Byte

The byte is the smallest data type among all the integer data types. It is an 8-bit signed two's complement integer. It stores whole numbers ranging from -128 to 127.

Syntax:

```
byte byteVariable;
```

Short

Short is a 16-bit signed two's complement integer. It stores whole numbers with values ranging from -32768 to 32767. Its default value is 0.

Syntax:

```
short shortVariable;
```

Primitive Data-Types (Float and Double Variables)

```
class FloatDataTypes
{
    public static void main(String args[]) {

        float f = 65.20298f;
        double d = 876.765d;

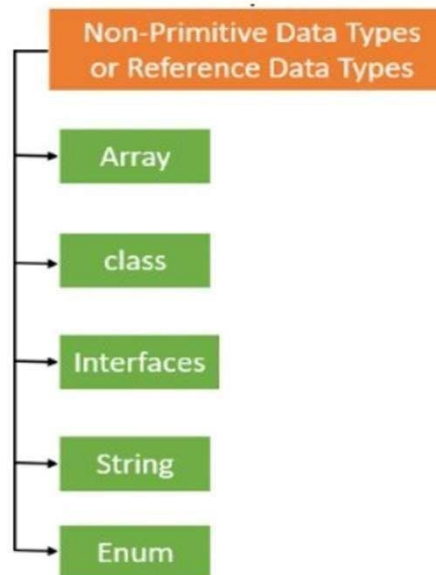
        System.out.println("The float variable is " + f);
        System.out.println("The double variable is " + d);
    }
}
```

Output:

```
The float variable is 65.20298
The double variable is 876.765
```

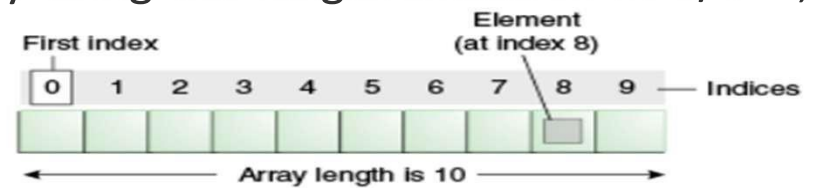
Non-Primitive Data-Types

- Non-primitive data types or reference data types refer to instances or objects.
- They cannot store the value of a variable directly in memory.
- They store a memory address of the variable.
- Unlike primitive data types we define by Java, non-primitive data types are user-defined.
- Programmers create them and can be assigned with null.



Non-Primitive Data-Types - Arrays

- An array is a collection of similar type of elements which has contiguous memory location.
- **Java array** is an object which contains elements of a similar data type. Additionally, The elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array.
- Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.
- Unlike C/C++, we can get the length of the array using the length member. In C/C++, we need to use the sizeof operator.
- `Int month_days[] = new int[12];`
- After this statement executes, month_days will refer to an array of 12 integers. Further, all elements in the array will be initialized to zero.
- There are two types of array.
 - Single Dimensional Array
 - Multidimensional Array



Non-Primitive Data-Types - Arrays - Initialization

- Array declaration, memory allocation and initialization separately.

```
6   int myArray[];           // array declaration
7   myArray = new int[3];    // memory allocation
8   myArray[0] = 3;         // initialization
9   myArray[1] = 5;
10  myArray[2] = 7;
11
12  for(int i=0;i<myArray.length;i++) //length is the property of array
13      System.out.println(myArray[i]); //printing the contents
```

- Memory allocation at the time of declaration

```
6   int myArray[] = new int[3]; // declaration and memory allocation
7   myArray[0] = 3;             // initialization
8   myArray[1] = 5;
9   myArray[2] = 7;
10
11  for(int i=0;i<myArray.length;i++) //length is the property of array
12      System.out.println(myArray[i]); //printing the contents
```

- Declaration, allocation and initialization at the same time.

```
6   int myArray[] = new int[]{3,5,7}; // declaration and memory allocation & initialization
7
8   for(int i=0;i<myArray.length;i++) //length is the property of array
9       System.out.println(myArray[i]); //printing the contents
10
```

```
6   int myArray[] = {3,5,7};
7
8   for(int i=0;i<myArray.length;i++)
9       System.out.println(myArray[i]);
```

Non-Primitive Data-Types - Arrays

- Obtaining an array is a two-step process.
 - First, you must declare a variable of the desired array type.
 - Second, you must allocate the memory that will hold the array, using `new`, and assign it to the array variable.
- Thus, in Java, all arrays are dynamically allocated.

Non-Primitive Data-Types - Arrays -1D

- Examples

```
1 class OnedimensionalStandard
2 {
3     public static void main(String args[])
4     {
5         int[] a=new int[3];//declaration
6         a[0]=10;//initialization
7         a[1]=20;
8         a[2]=30;
9         //printing array
10        System.out.println("One dimensional array elements are");
11        System.out.println(a[0]);
12        System.out.println(a[1]);
13        System.out.println(a[2]);
14    }
15 }
```

Using For Loop – One Dimensional Array

```
1 class OnedimensionalLoop
2 {
3     public static void main(String args[])
4     {
5         int a[]={10,20,30,40,50};//declaration and initialization
6         System.out.println("One dimensional array elements are :\n");
7         for(int i=0;i<a.length;i++)
8         {
9             System.out.println("a["+i+"]:"+a[i]);
10        }
11    }
12 }
```

Output:

```
1 One dimensional array elements are :
2
3 a[0]:10
4 a[1]:20
5 a[2]:30
6 a[3]:40
7 a[4]:50
```

```
1 import java.util.*;
2 class OnedimensionalScanner
3 {
4     public static void main(String args[])
5     {
6         int len;
7         Scanner sc=new Scanner(System.in);
8         System.out.println("Enter Array length : ");
9         len=sc.nextInt();
10        int a[]=new int[len];//declaration
11        System.out.print("Enter " + len + " Element to Store in Array :\n");
12        for(int i=0; i<len; i++)
13        {
14            a[i] = sc.nextInt();
15        }
16        System.out.print("Elements in Array are :\n");
17        for(int i=0; i<len; i++)
18        {
19            System.out.print(a[i] + " ");
20        }
21    }
22 }
```

Output:

```
1 Enter Array length :
2 4
3 Enter 4 Element to Store in Array :
4 1
5 2
6 3
7 4
8 Elements in Array are :
9 1 2 3 4
```

Non-Primitive Data-Types - Arrays -1D

- One Dimensional Array Examples

Putting together all the pieces, here is a program that creates an array of the number of days in each month:

```
// Demonstrate a one-dimensional array.
```

```
class Array {  
    public static void main(String args[]) {  
        int month_days[];  
        month_days = new int[12];  
        month_days[0] = 31;  
        month_days[1] = 28;  
        month_days[2] = 31;  
        month_days[3] = 30;  
        month_days[4] = 31;  
        month_days[5] = 30;  
        month_days[6] = 31;  
        month_days[7] = 31;  
        month_days[8] = 30;  
        month_days[9] = 31;  
        month_days[10] = 30;  
        month_days[11] = 31;  
        System.out.println("April has " + month_days[3] + " days.");  
    }  
}
```

```
// An improved version of the previous program.
```

```
class AutoArray {  
    public static void main(String args[]) {  
        int month_days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31,  
                               30, 31 };  
        System.out.println("April has " + month_days[3] + " days.");  
    }  
}
```

Non-Primitive Data-Types - Arrays -1D

- One Dimensional Array Examples

```
// An improved version of the previous program.
class AutoArray {
    public static void main(String args[]) {

        int month_days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31,
                             30, 31 };
        System.out.println("April has " + month_days[3] + " days.");
    }
}
```

Non-Primitive Data-Types - Arrays -1D

- One Dimensional Array Examples
 - Average an array of values

```
// Average an array of values.  
class Average {  
    public static void main(String args[]) {  
        double nums[] = {10.1, 11.2, 12.3, 13.4, 14.5};  
        double result = 0;  
        int i;  
        for(i=0; i<5; i++)  
            result = result + nums[i];  
        System.out.println("Average is " + result / 5);  
    }  
}
```

Non-Primitive Data-Types - Arrays -1D

- One Dimensional Array Examples

Example 2: Sum of Array Elements

java

```
public class OneDimensionalArrayExample2 {  
    public static void main(String[] args) {  
        // Declare and initialize an array of integers  
        int[] numbers = {2, 4, 6, 8, 10};  
  
        // Calculate the sum of array elements  
        int sum = 0;  
        for (int i = 0; i < numbers.length; i++) {  
            sum += numbers[i];  
        }  
  
        // Display the sum  
        System.out.println("Sum of array elements: " + sum);  
    }  
}
```


Non-Primitive Data-Types - Arrays -1D

- One Dimensional Array Examples

Example 3: Finding Maximum Element

java

```
public class OneDimensionalArrayExample3 {  
    public static void main(String[] args) {  
        // Declare and initialize an array of integers  
        int[] numbers = {14, 7, 21, 35, 10};  
  
        // Find the maximum element in the array  
        int max = numbers[0];  
        for (int i = 1; i < numbers.length; i++) {  
            if (numbers[i] > max) {  
                max = numbers[i];  
            }  
        }  
  
        // Display the maximum element  
        System.out.println("Maximum element in the array: " + max);  
    }  
}
```

Non-Primitive Data-Types - Arrays -1D

- One Dimensional Array Examples

- String Array

```
public class OneDimensionalArrayExample4 {  
    public static void main(String[] args) {  
        // Declare and initialize an array of strings  
        String[] fruits = {"Apple", "Banana", "Orange", "Mango", "Grapes"};  
  
        // Access and print elements  
        System.out.println("Fruits in the array:");  
        for (int i = 0; i < fruits.length; i++) {  
            System.out.println("Fruit at index " + i + ": " + fruits[i]);  
        }  
    }  
}
```

Non-Primitive Data-Types - Arrays -2D

- Two Dimensional Array Examples

Representation of 2D array in Tabular Format:

A two – dimensional array can be seen as a table with 'x' rows and 'y' columns where the row number ranges from 0 to (x-1) and column number ranges from 0 to (y-1). A two – dimensional array 'x' with 3 rows and 3 columns is shown below:

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

Non-Primitive Data-Types - Arrays -2D

- Two Dimensional Array Examples : To print the 2d array in tabular format



```
import java.io.*;
class GFG {
    public static void main(String[] args)
    {
        int[][] arr = { { 1, 2 }, { 3, 4 } };

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                System.out.print(arr[i][j] + " ");
            }

            System.out.println();
        }
    }
}
```

Output

```
1 2
3 4
```

Non-Primitive Data-Types - Arrays -2D

- Two Dimensional Array Examples

```
import java.io.*;
public class GFG {
    public static void main(String[] args)
    {

        int[][] arr = new int[10][20];
        arr[0][0] = 1;

        System.out.println("arr[0][0] = " + arr[0][0]);

    }
}
```

```
import java.io.*;
class GFG {
    public static void main(String[] args)
    {

        int[][] arr = { { 1, 2 }, { 3, 4 } };

        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                System.out.println("arr[" + i + "][" + j + "] = "
                                    + arr[i][j]);

    }
}
```

Output

```
arr[0][0] = 1
arr[0][1] = 2
arr[1][0] = 3
arr[1][1] = 4
```

Non-Primitive Data-Types - Arrays -2D

- Two Dimensional Array Examples

```
1 //Initializing and Accessing Elements
2
3 public class TwoDimensionalArrayExample1 {
4     public static void main(String[] args) {
5         // Declare and initialize a 2D array of integers
6         int[][] matrix = {
7             {1, 2, 3},
8             {4, 5, 6},
9             {7, 8, 9}
10        };
11
12        // Access and print elements
13        System.out.println("Elements of the 2D array:");
14        for (int i = 0; i < matrix.length; i++) {
15            for (int j = 0; j < matrix[i].length; j++) {
16                System.out.print(matrix[i][j] + " ");
17            }
18            System.out.println(); // Move to the next line for a new row
19        }
20    }
21 }
22
```

```
1 // SUM OF MATRIX ELEMENTS
2
3 public class TwoDimensionalArrayExample2 {
4     public static void main(String[] args) {
5         // Declare and initialize a 2D array of integers
6         int[][] matrix = {
7             {2, 4, 6},
8             {8, 10, 12},
9             {14, 16, 18}
10        };
11
12        // Calculate the sum of matrix elements
13        int sum = 0;
14        for (int i = 0; i < matrix.length; i++) {
15            for (int j = 0; j < matrix[i].length; j++) {
16                sum += matrix[i][j];
17            }
18        }
19
20        // Display the sum
21        System.out.println("Sum of matrix elements: " + sum);
22    }
23 }
24
```


Non-Primitive Data-Types - Arrays -2D

- Two Dimensional Array Examples

```
1 //Finding Maximum Element in Matrix
2
3 public class TwoDimensionalArrayExample3 {
4     public static void main(String[] args) {
5         // Declare and initialize a 2D array of integers
6         int[][] matrix = {
7             {15, 7, 23},
8             {10, 21, 35},
9             {18, 12, 27}
10        };
11
12        // Find the maximum element in the matrix
13        int max = matrix[0][0];
14        for (int i = 0; i < matrix.length; i++) {
15            for (int j = 0; j < matrix[i].length; j++) {
16                if (matrix[i][j] > max) {
17                    max = matrix[i][j];
18                }
19            }
20        }
21
22        // Display the maximum element
23        System.out.println("Maximum element in the matrix: " + max);
24    }
25 }
26
```

```
1 // STRING 2D Array
2
3 public class TwoDimensionalArrayExample4 {
4     public static void main(String[] args) {
5         // Declare and initialize a 2D array of strings
6         String[][] names = {
7             {"John", "Alice", "Bob"},
8             {"Mary", "David", "Eva"},
9             {"Tom", "Olivia", "Charlie"}
10        };
11
12        // Access and print elements
13        System.out.println("Names in the 2D array:");
14        for (int i = 0; i < names.length; i++) {
15            for (int j = 0; j < names[i].length; j++) {
16                System.out.print(names[i][j] + " ");
17            }
18            // Move to the next line for a new row
19            System.out.println();
20        }
21    }
22 }
23
```

Non-Primitive Data-Types - Arrays -2D

Two Dimensional Array Examples – Addition of Two matrices- Hardcoded Inputs.

```
1 public class MatrixAdditionHardcoded {
2
3     public static void main(String[] args) {
4         // Hardcoded input values for the matrices
5         int[][] matrix1 = {
6             {1, 2, 3},
7             {4, 5, 6},
8             {7, 8, 9}
9         };
10
11         int[][] matrix2 = {
12             {9, 8, 7},
13             {6, 5, 4},
14             {3, 2, 1}
15         };
16
17         // Add the matrices
18         int rows = matrix1.length;
19         int columns = matrix1[0].length;
20         int[][] resultMatrix = new int[rows][columns];
21         for (int i = 0; i < rows; i++) {
22             for (int j = 0; j < columns; j++) {
23                 resultMatrix[i][j] = matrix1[i][j] + matrix2[i][j];
24             }
25         }
26
27         // Display the result matrix (sum of matrices)
28         System.out.println("Matrix 1:");
29         displayMatrix(matrix1);
30
31         System.out.println("Matrix 2:");
32         displayMatrix(matrix2);
33
34         System.out.println("Resultant matrix (Sum of matrices):");
35         displayMatrix(resultMatrix);
36     }
}
```



MatrixAdditionHardcoded.java

```
37
38 // Helper method to display a matrix
39 private static void displayMatrix(int[][] matrix) {
40     for (int i = 0; i < matrix.length; i++) {
41         for (int j = 0; j < matrix[i].length; j++) {
42             System.out.print(matrix[i][j] + " ");
43         }
44         System.out.println(); // Move to the next line for a new row
45     }
46     System.out.println(); // Add a blank line between matrices
47 }
48 }
49
```


Java Methods

- The **method in Java** or Methods of Java is a collection of statements that perform some specific task and return the result to the caller. A Java method can perform some specific task without returning anything. Java Methods allow us to **reuse** the code without retyping the code. In Java, every method must be part of some class that is different from languages like C, C++, and Python.

<https://www.geeksforgeeks.org/methods-in-java/?ref=lbp>



java_methods.pdf

Operators - Types

- There are multiple types of operators in Java all are mentioned below:
- [Arithmetic Operators](#)
- [Unary Operators](#)
- [Assignment Operator](#)
- [Relational Operators](#)
- [Logical Operators](#)
- [Ternary Operator](#)
- [Bitwise Operators](#)
- [Shift Operators](#)
- [instance of operator](#)

Operators – Arithmetic Operators

A list of all arithmetic operators:

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	$++x$
--	Decrement	Decreases the value of a variable by 1	$--x$

Example 1: Arithmetic Operators

```
class Main {  
    public static void main(String[] args) {  
  
        // declare variables  
        int a = 12, b = 5;  
  
        // addition operator  
        System.out.println("a + b = " + (a + b));  
  
        // subtraction operator  
        System.out.println("a - b = " + (a - b));  
  
        // multiplication operator  
        System.out.println("a * b = " + (a * b));  
  
        // division operator  
        System.out.println("a / b = " + (a / b));  
  
        // modulo operator  
        System.out.println("a % b = " + (a % b));  
    }  
}
```

Output

```
a + b = 17  
a - b = 7  
a * b = 60  
a / b = 2  
a % b = 2
```

Operators – Assignment Operators

A list of all assignment operators:

Example 2: Assignment Operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

```
class Main {  
    public static void main(String[] args) {  
  
        // create variables  
        int a = 4;  
        int var;  
  
        // assign value using =  
        var = a;  
        System.out.println("var using = is " + var);  
  
        // assign value using +=  
        var += a;  
        System.out.println("var using += is " + var);  
  
        // assign value using *=  
        var *= a;  
        System.out.println("var using *= is " + var);  
    }  
}
```

Output

```
var using = is 4  
var using += is 8  
var using *= is 32
```

Operators – Relational Operators

A list of all relational operators:

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Example 3: Relational Operators

```
class Main {  
    public static void main(String[] args) {  
  
        // create variables  
        int a = 7, b = 11;  
  
        // value of a and b  
        System.out.println("a is " + a + " and b is " + b);  
  
        // == operator  
        System.out.println(a == b); // false  
  
        // != operator  
        System.out.println(a != b); // true  
  
        // > operator  
        System.out.println(a > b); // false  
  
        // < operator  
        System.out.println(a < b); // true  
  
        // >= operator  
        System.out.println(a >= b); // false  
  
        // <= operator  
        System.out.println(a <= b); // true  
    }  
}
```

Operators – Logical Operators

4. Java Logical Operators

Logical operators are also used to check whether an expression is `true` or `false`. They too are used in decision-making.

A list of all logical operators:

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	<code>x < 5 && x < 10</code>
	Logical or	Returns true if one of the statements is true	<code>x < 5 x < 4</code>
!	Logical not	Reverse the result, returns false if the result is true	<code>!(x < 5 && x < 10)</code>

Example 4: Logical Operators

```
class Main {  
    public static void main(String[] args) {  
  
        // && operator  
        System.out.println((5 > 3) && (8 > 5)); // true  
        System.out.println((5 > 3) && (8 < 5)); // false  
  
        // || operator  
        System.out.println((5 < 3) || (8 > 5)); // true  
        System.out.println((5 > 3) || (8 < 5)); // true  
        System.out.println((5 < 3) || (8 < 5)); // false  
  
        // ! operator  
        System.out.println(!(5 == 3)); // true  
        System.out.println(!(5 > 3)); // false  
    }  
}
```

Operators – Unary Operators

- Unary Minus (-)
- NOT operator (!)
- Increment Operator (++)
 - Post Increment
 - Pre Increment
- Decrement Operator (--)
 - Post Increment
 - Pre Increment
- Bitwise Complement (~)

<https://www.geeksforgeeks.org/java-unary-operator-with-examples/>

Operators – Ternary Operators

- Java ternary operator is the only conditional operator that takes three operands.
- It's a one-liner replacement for the if-then-else statement and is used a lot in Java programming.

Syntax:

```
variable = Expression1 ? Expression2: Expression3
```

If operates similarly to that of the if-else statement as in *Expression2* is executed if *Expression1* is true else *Expression3* is executed.

```
if(Expression1)
{
    variable = Expression2;
}
else
{
    variable = Expression3;
}
```

Example:

```
num1 = 10;
num2 = 20;
```

```
res=(num1>num2) ? (num1+num2):(num1-num2)
```

Since num1<num2,
the second operation is performed
res = num1-num2 = -10

Operators – Ternary Operators

- Java ternary operator is the only conditional operator that takes three operands.

```
// Java program to find largest among two
// numbers using ternary operator

import java.io.*;

class Ternary {
    public static void main(String[] args)
    {

        // variable declaration
        int n1 = 5, n2 = 10, max;

        System.out.println("First num: " + n1);
        System.out.println("Second num: " + n2);

        // Largest among n1 and n2
        max = (n1 > n2) ? n1 : n2;

        // Print the largest number
        System.out.println("Maximum is = " + max);
    }
}
```

Operators – Ternary Operators

- Java ternary operator is the only conditional operator that takes three operands.

```
// Java program to find largest among two
// numbers using ternary operator

import java.io.*;

class Ternary {
    public static void main(String[] args)
    {

        // variable declaration
        int n1 = 5, n2 = 10, max;

        System.out.println("First num: " + n1);
        System.out.println("Second num: " + n2);

        // Largest among n1 and n2
        max = (n1 > n2) ? n1 : n2;

        // Print the largest number
        System.out.println("Maximum is = " + max);
    }
}
```

Output

```
First num: 5
Second num: 10
Maximum is = 10
```

Operators – Shift Operators

- Left Shift <<
- Signed Right Shift >>
- Unsigned Right Shift >>>

Operators – Shift Operators **Left Shift <<**

- Left shifting a number by certain positions is equivalent to multiplying the number by two raised to the power of the specified positions. That is,

```
left shift x by n positions <=> x * 2n
```

Syntax:

```
x << n
```

Here,

x: an integer

n: a non-negative integer

Binary	Decimal
1 0 1	5
↓ Left shift once	
1 0 1 0	10
↓ Left shift twice	
1 0 1 0 0 0	40

Operators – Shift Operators

Left Shift <<

Binary equivalent of 5 is 0101.

1	0	1
---	---	---

Assume that the statement is as follows:

$x \ll 4$, let y be 4

Assume that the statement is as follows

$x \ll 4$:

Let y be 4.

At y=1 (Shifting the first time to left)

1	0	1	0
---	---	---	---

← Zero added at the end on shifting digits to left.

It is equivalent to 10.

At y = 2

1	0	1	0	0
---	---	---	---	---

← Zero appended

It is equivalent to 20.

At y = 3

1	0	1	0	0	0
---	---	---	---	---	---

← Zero appended

It is equivalent to 40.

At y = 4

1	0	1	0	0	0	0
---	---	---	---	---	---	---

← Zero appended resulting to final answer

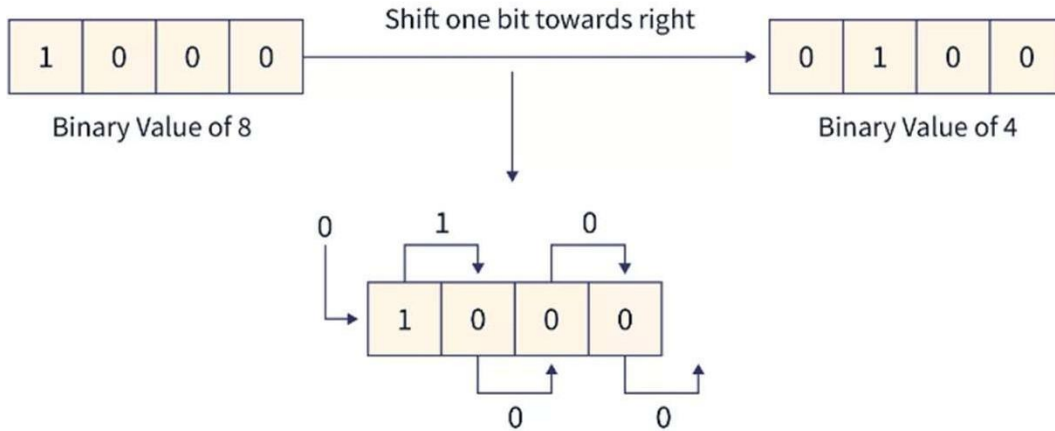
It is equivalent to 80.

LeftShift.java

```
public class LeftShift {  
    public static void main( String[ ] args ) {  
        int x = 5 ;  
        // shifting the value by 4 using left shift operator  
        int y = x << 4 ;  
        // printing the left shifted value  
        System.out.println( " The value is: " +y) ;  
    }  
}
```

Operators – Shift Operators **Signed Right Shift >>**

- When a bit is shifted towards the right, the rightmost bit (least significant bit) is discarded and the leftmost bit (most significant) bit is filled with the sign bit (i.e.) when the number is positive, 0 is used to fill the leftmost position, and when the number is negative, the sign bit is used to fill the leftmost position.



- he right in the binary value of 8. When we shift a bit towards the right, the rightmost bit (least significant bit) is discarded, and the leftmost bit (most significant bit) is filled with a sign bit (0 because 8 is a positive number). So the output became 0100, the binary value of 4.

<https://www.scaler.com/topics/right-shift-operator-in-java/>

Operators – Shift Operators **Signed Right Shift >>**

Example 1: Signed Right Shift Operator for a Positive Number

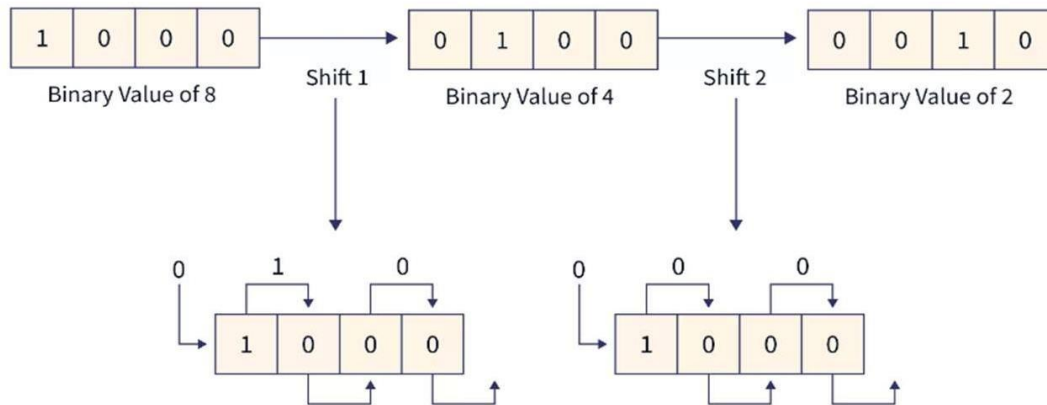
Let's take a positive number (8) and shift its bits twice ($8 \gg 2$) using the signed right shift operator.

Solution

The binary value of 8 is 1000, and we need to shift its bits twice towards the right.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("8 >> 2: " + (8 >> 2));  
    }  
}
```

The below image illustrates the step-by-step shifting for $8 \gg 2$



Output

```
8 >> 2: 2
```

Java Control-Flow Statements

Java Control Flow Statements Cheat Sheet			Java Concept Of The Day
What are control flow statements? Control flow statements are the statements in a program which control the flow of execution of a program. There are three types of control flow statements in Java. 1. Decision Making Statements 2. Looping Statements 3. Branching Statements			
Decision Making Statements Decision making statements determine which statements to execute depending upon the outcome of a supplied condition or an expression. There are two different decision making statements available in Java. 1. if statements 2. switch statement 1) if Statements a) Simple if Statement <pre>if (condition) { //Set of statements //These are executed only if condition is TRUE }</pre> b) if-else statement <pre>if (condition) { //Set of statements //These are executed when condition is TRUE } else { //Set of statements //These are executed when condition is FALSE }</pre> c) if-else-if Statement <pre>if (condition_1) { //Set of statements //These are executed if condition_1 is TRUE } else if (condition_2) { //Set of statements //These are executed if condition_2 is TRUE } else { //Set of statements //These are executed if all conditions are FALSE }</pre> 2) switch Statement <pre>switch (key) { case value_1 : Statement_1; break; case value_2 : Statement_2; break; . . . default : Default_Statement; break; }</pre>	Looping Statements Looping statements are used to execute a set of statements repeatedly until supplied condition becomes FALSE. There are four types of loop statements in Java. 1. for loop 2. while loop 3. do-while loop 4. for-each loop 1) for Loop <pre>for (initialization; termination; increment / decrement) { //Set of statements }</pre> 2) while loop <pre>while (condition) { //Set of statements //These are executed until condition is TRUE }</pre> 3) do-while loop <pre>do { //Set of statements //These are executed until condition is TRUE } while (condition);</pre> 4) for-each loop / advanced for loop <pre>for (data_type variable_Name : array / collection) { //Set of statements }</pre>	Branching Statements Branching statements or jump statements are used to transfer the control of execution to some other part of the program. There are three branching statements in Java. 1. break statement 2. continue statement 3. return statement 1) break statement <pre>while (condition) { //Some statements if (condition to break) { break; } //Some statements }</pre> 2) continue Statement <pre>while (condition) { //Some statements if (condition to continue) { continue; } //Some statements }</pre> 3) return Statement <pre>return_Type anymethod(Pass_arguments_Here) { //Some statements return anything or Nothing; }</pre>	

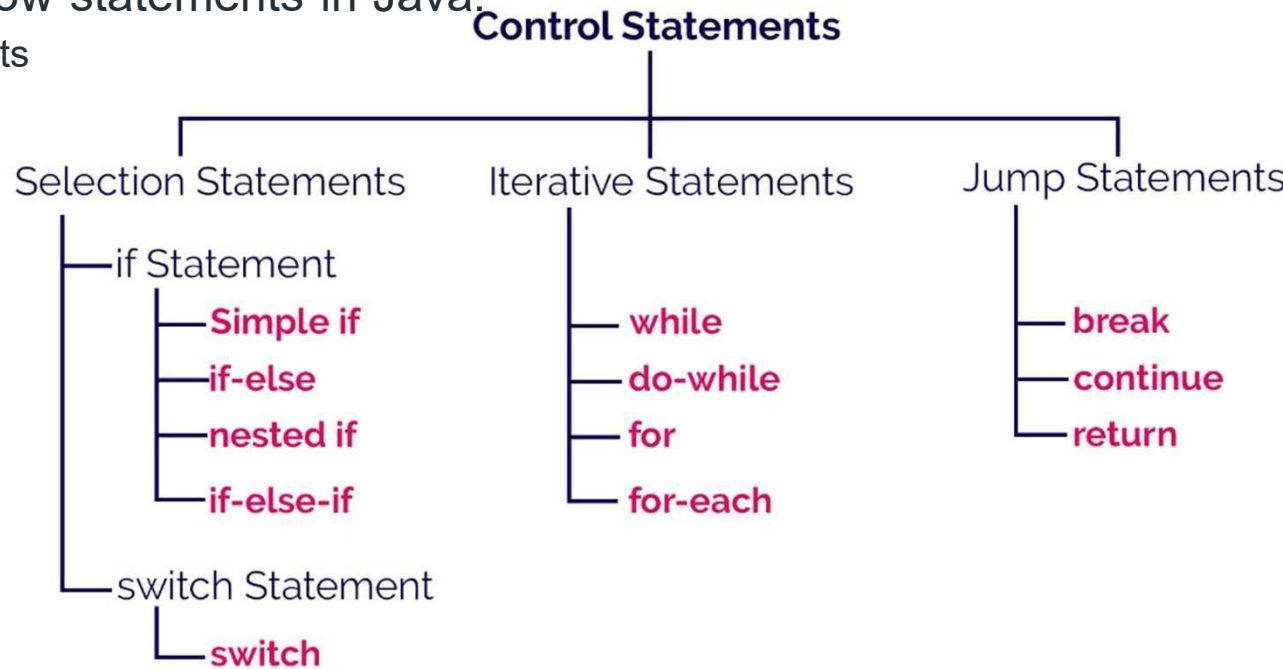
Java Control Flow Statements

- **What are control flow statements?**

- Control flow statements are the statements in a program which control the flow of execution of a program.

- There are three types of control flow statements in Java.

- Decision Making/Selection Statements
- Looping/Iterative Statements
- Branching/Jump Statements



<https://www.geeksforgeeks.org/decision-making-javaif-else-switch-break-continue-jump/>

Java Control Flow Statements – Decision making Statements - if

a) Simple if Statement

Simple if statement is the most basic of all control flow statements. It is used when you want a certain block of code or set of statements are to be executed only when a supplied condition evaluates to TRUE.

```
1  if (condition)
2  {
3      //Set of statements : These are executed only if condition is TRUE
4  }
```

b) if-else statement

It is an extension to simple if statement. It has two blocks. One is if-block and another one is else-block. if-block is executed if condition evaluates to TRUE and else-block is executed if condition is evaluated to FALSE.

```
1  if (condition)
2  {
3      //Set of statements : These are executed when condition is TRUE
4  }
5  else
6  {
7      //Set of statements : These are executed when condition is FALSE
8  }
```

Java Control Flow Statements – Decision making Statements - if

c) if-else-if Statement

if-else-if statement is used when more than one conditions are to be evaluated. If a particular condition evaluates to TRUE, block associated with that condition will be executed. If all the conditions are evaluated to FALSE, else-block will be executed at the end.

```
1  if (condition_1)
2  {
3      //Set of statements : These are executed if condition_1 is TRUE
4  }
5  else if (condition_2)
6  {
7      //Set of statements : These are executed if condition_2 is TRUE
8  }
9  else
10 {
11     //Set of statements : These are executed if all conditions are FALSE
12 }
```

Java Control Flow Statements – Decision making Statements – if Nested-if and if-else ladder examples

```
// Java program to illustrate nested-if statement
import java.util.*;

class NestedIfDemo {
    public static void main(String args[])
    {
        int i = 10;

        if (i == 10 || i < 15) {
            // First if statement
            if (i < 15)
                System.out.println("i is smaller than 15");

            // Nested - if statement
            // Will only be executed if statement above
            // it is true
            if (i < 12)
                System.out.println(
                    "i is smaller than 12 too");
        } else{
            System.out.println("i is greater than 15");
        }
    }
}
```

```
// Java program to illustrate if-else-if ladder
import java.util.*;

class ifelseifDemo {
    public static void main(String args[])
    {
        int i = 20;

        if (i == 10)
            System.out.println("i is 10");
        else if (i == 15)
            System.out.println("i is 15");
        else if (i == 20)
            System.out.println("i is 20");
        else
            System.out.println("i is not present");
    }
}
```

Java Control Flow Statements – Decision making Statements – **switch Statement**

The `switch` statement allows us to execute a block of code among many alternatives.

The syntax of the `switch` statement in Java is:

```
switch (expression) {  
    case value1:  
        // code  
        break;  
  
    case value2:  
        // code  
        break;  
  
    ...  
    ...  
  
    default:  
        // default statements  
}
```

How does the switch-case statement work?

The `expression` is evaluated once and compared with the values of each case.

- If `expression` matches with `value1`, the code of `case value1` are executed. Similarly, the code of `case value2` is executed if `expression` matches with `value2`.
- If there is no match, the code of the **default case** is executed.

We use the **break** keyword to terminate the code whenever a certain condition is met (when the expression matches with a case).

`switch` works with the expression output which must be either **or short or int or enum types or string literals**

Java Control Flow Statements – Decision making Statements – **switch** Statement - Examples

We can use a switch case statement to convert a numerical representation of a month into its corresponding string representation in Java

```
1 import java.util.Scanner;
2
3 public class MonthConverter {
4     public static void main(String[] args) {
5         // Create a Scanner object to read input from the user
6         Scanner scanner = new Scanner(System.in);
7
8         // Prompt the user to enter a number representing a month
9         System.out.print("Enter a number (1-12) representing a month: ");
10        int monthNumber = scanner.nextInt();
11
12        // Close the Scanner to avoid resource leak
13        scanner.close();
14
15        // Convert the month number to a month string using switch case
16        String monthString;
17        switch (monthNumber) {
18            case 1:
19                monthString = "January";
20                break;
21            case 2:
22                monthString = "February";
23                break;
24            case 3:
25                monthString = "March";
26                break;
27            case 4:
28                monthString = "April";
29                break;
30            case 5:
31                monthString = "May";
32                break;
33            case 6:
34                monthString = "June";
35                break;
36            case 7:
37                monthString = "July";
38                break;
39            case 8:
40                monthString = "August";
41                break;
```



MonthConverter.java

```
42        case 9:
43            monthString = "September";
44            break;
45        case 10:
46            monthString = "October";
47            break;
48        case 11:
49            monthString = "November";
50            break;
51        case 12:
52            monthString = "December";
53            break;
54        default:
55            monthString = "Invalid month number";
56            break;
57    }
58
59    // Display the result
60    System.out.println("The month corresponding to the number " + monthNumber +
61                      " is: " + monthString);
62 }
63 }
```

Java Control Flow Statements – Decision making Statements – **switch**

Statement - Examples

The following example illustrates the multiple cases in a single block. The below example calculates the number of days in a particular month.

```
65 class SwitchDemo2 {
66     public static void main(String[] args) {
67
68         int month = 2;
69         int year = 2000;
70         int numDays = 0;
71
72         switch (month) {
73             case 1: case 3: case 5:
74             case 7: case 8: case 10:
75             case 12:
76                 numDays = 31;
77                 break;
78             case 4: case 6:
79             case 9: case 11:
80                 numDays = 30;
81                 break;
82             case 2:
83                 if (((year % 4 == 0) &&
84                     !(year % 100 == 0))
85                     || (year % 400 == 0))
86                     numDays = 29;
87                 else
88                     numDays = 28;
89                 break;
90             default:
91                 System.out.println("Invalid month.");
92                 break;
93         }
94         System.out.println("Number of Days = "
95                             + numDays);
96     }
97 }
```



SwitchDemo2.java

Java Control Flow Statements – Decision making Statements – switch Statement - Examples



StringSwitchDemo.java

Using Strings in switch Statements. The following example **StringSwitchDemo**, displays the number of the month based on the value of the String named month:

```
1 public class StringSwitchDemo {
2
3     public static int getMonthNumber(String month) {
4
5         int monthNumber = 0;
6
7         if (month == null) {
8             return monthNumber;
9         }
10
11         switch (month.toLowerCase()) {
12             case "january":
13                 monthNumber = 1;
14                 break;
15             case "february":
16                 monthNumber = 2;
17                 break;
18             case "march":
19                 monthNumber = 3;
20                 break;
21             case "april":
22                 monthNumber = 4;
23                 break;
24             case "may":
25                 monthNumber = 5;
26                 break;
27             case "june":
28                 monthNumber = 6;
29                 break;
30             case "july":
31                 monthNumber = 7;
32                 break;
33             case "august":
34                 monthNumber = 8;
35                 break;
36             case "september":
37                 monthNumber = 9;
38                 break;
39             case "october":
40                 monthNumber = 10;
41                 break;
42             case "november":
43                 monthNumber = 11;
44                 break;
45             case "december":
46                 monthNumber = 12;
47                 break;
48             default:
49                 monthNumber = 0;
50                 break;
51         }
52         return monthNumber;
53     }
54 }
55
56 public static void main(String[] args) {
57
58     String month = "August";
59
60     int returnedMonthNumber =
61         StringSwitchDemo.getMonthNumber(month);
62
63     if (returnedMonthNumber == 0) {
64         System.out.println("Invalid month");
65     } else {
66         System.out.println(returnedMonthNumber);
67     }
68 }
69 }
```


Java Control Flow Statements – Decision making Statements – switch Statement - Examples

Using Strings in switch Statements. The following is an another example which shows usage of strings in switch-case statement

```
class Username {  
    public static void main(String[] args) {  
  
        String username = "Ihechikara";  
  
        switch (username) {  
            case "Doe":  
                System.out.println("Username is Doe");  
                break;  
            case "John":  
                System.out.println("Username is John");  
                break;  
            case "Jane":  
                System.out.println("Username is Jane");  
                break;  
            default:  
                System.out.println("Username not found!");  
                // Username not found!  
        }  
    }  
}
```

Java Control Flow Statements – Iterative/Loop Statements

1) for Loop

for statement or for loop provides way to specify initialization, termination condition and increment/decrement in a single statement. It is often used to iterate over an array or a collection.

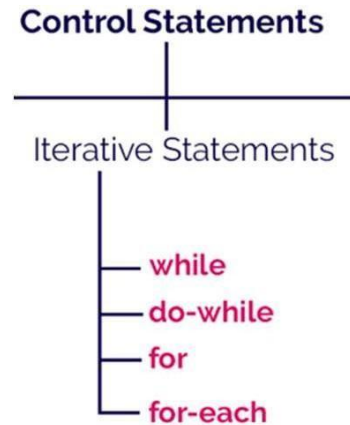
```
1 | for (initialization; termination; increment/decrement)
2 | {
3 |     //Set Of Statements
4 | }
```

2) while loop

In while loop, a set of statements are executed repeatedly until condition is TRUE. If the condition becomes FALSE, control goes out of while loop. Use this loop when you don't know exact number of iterations.

This loop is entry controlled loop as condition is checked while entering the loop.

```
1 | while (condition)
2 | {
3 |     //Set of statements : These are executed until condition is TRUE
4 | }
```

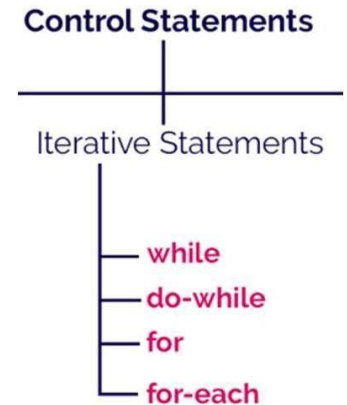


Java Control Flow Statements – Iterative/Loop Statements

3) do-while loop

It is same as while loop, but here condition is checked while exiting the loop. Hence, this loop is exit controlled loop.

```
1  do
2  {
3      //Set of statements : These are executed until condition is TRUE
4  }
5  while (condition);
```



4) for-each loop or advanced for loop

It is used to iterate all the elements of an array or a collection at a time. You can't iterate only few elements of a collection/array using this loop. This is the main difference between basic for loop and advanced for loop.

```
1  for (data_type variable_Name : array/collection)
2  {
3      //Set of statements
4  }
```

Java Control Flow Statements – Iterative/Loop Statements

while Statement

- The while statement evaluates expression, which must return a boolean value.
- If the expression evaluates to true, the while statement executes the statement(s) in the while block.
- The while statement continues testing the expression and executing its block until the expression evaluates to false.
- Using the while statement to print the values from 1 through 10 can be accomplished as in the following WhileDemo program:

```
1  class WhileDemo {  
2  
3      public static void main(String[] args){  
4          int count = 1;  
5          while (count < 11) {  
6              System.out.println("Count is: " + count);  
7              count++;  
8          }  
9      }  
10 }  
11
```

Java Control Flow Statements – Iterative/Loop Statements

do-while Statement

- The difference between do-while and while is that do-while evaluates its expression at the bottom of the loop instead of the top.
- Therefore, the statements within the do block are always executed at least once, as shown in the following DoWhileDemo program:

```
1  class DoWhileDemo {  
2      public static void main(String[] args){  
3          int count = 1;  
4          do {  
5              System.out.println("Count is: " + count);  
6              count++;  
7          } while (count < 11);  
8      }  
9  }
```

Java Control Flow Statements – Iterative/Loop Statements

for Statement

- The for statement provides a compact way to iterate over a range of values.
- When using this version of the for statement, keep in mind that:
 - The **initialization expression** initializes the loop; it's executed once, as the loop begins.
 - When the **termination expression** evaluates to false, the loop terminates.
 - The **increment expression** is invoked after each iteration through the loop; it is perfectly acceptable for this expression to increment or decrement a value.

```
1 class ForDemo {  
2     public static void main(String[] args){  
3         for(int i=1; i<11; i++){  
4             System.out.println("Count is: " + i);  
5         }  
6     }  
7 }  
-
```

The output of this program is:

```
Count is: 1  
Count is: 2  
Count is: 3  
Count is: 4  
Count is: 5  
Count is: 6  
Count is: 7  
Count is: 8  
Count is: 9  
Count is: 10
```

Java Control Flow Statements – Iterative/Loop Statements

for–each (enhanced for) Statement

- The for statement also has another form designed for iteration through Collections and arrays. This form is sometimes referred to as the **enhanced for statement**, and can be used to **make your loops more compact and easy to read**.
- The following program, *EnhancedForDemo*, uses the enhanced for to loop through the array.

```
1  class EnhancedForDemo {  
2  
3      public static void main(String[] args){  
4  
5          int[] numbers = {1,2,3,4,5,6,7,8,9,10};  
6  
7          for (int item : numbers) {  
8  
9              System.out.println("Count is: " + item);  
10         }  
11     }  
12 }  
13
```

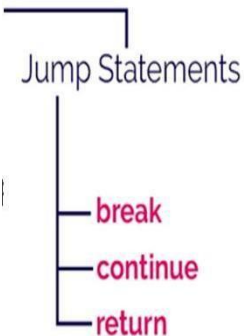
In this example, the variable `item` holds the current value from the `numbers` array. The output from this program is the same as before:

```
Count is: 1  
Count is: 2  
Count is: 3  
Count is: 4  
Count is: 5  
Count is: 6  
Count is: 7  
Count is: 8  
Count is: 9  
Count is: 10
```

Java Control Flow Statements – Jump/Branching Statements - break

- **Break Statement**

- **Un-labelled break** : used to terminate immediate loop i.e inner most loop
- **Labelled break** : used to terminate outer loops using the label.
- The below program illustrates the usage of **un-labelled break statement**. The program searches for a number 12 in an 1-d array. The break statement is used to terminate the for loop when that value is found. Control-flow transfer the control after the for loop.



```
1 class BreakDemo_UnLabelled {
2     public static void main(String[] args) {
3
4         int[] arrayOfInts = { 32, 87, 3, 589, 12, 1076, 2000, 8, 622, 127 };
5
6         int searchfor = 12;
7
8         int i;
9         boolean foundIt = false;
10
11         for (i = 0; i < arrayOfInts.length; i++) {
12             if (arrayOfInts[i] == searchfor) {
13                 foundIt = true;
14                 break;
15             }
16         }
17
18         if (foundIt) {
19             System.out.println("Found " + searchfor + " at index " + i);
20         } else {
21             System.out.println(searchfor + " not in the array");
22         }
23     }
24 }
```


Java Control Flow Statements – Jump/Branching Statements - break

- The below program illustrates the usage of **labelled break statement**. The following program, BreakWithLabelDemo, is similar to the previous program, but uses nested for loops to search for a value in a **two-dimensional array**. When the value is found, a labeled break terminates the outer for loop (labeled "search"):

```
1  class BreakWithLabelDemo {
2      public static void main(String[] args) {
3
4          int[][] arrayOfInts = {
5              { 32, 87, 3, 589 },
6              { 12, 1076, 2000, 8 },
7              { 622, 127, 77, 955 }
8          };
9          int searchfor = 12;
10
11         int i;
12         int j = 0;
13         boolean foundIt = false;
14
15         search:
16         for (i = 0; i < arrayOfInts.length; i++) {
17             for (j = 0; j < arrayOfInts[i].length; j++) {
18                 if (arrayOfInts[i][j] == searchfor) {
19                     foundIt = true;
20                     break search;
21                 }
22             }
23         }
24
25         if (foundIt) {
26             System.out.println("Found " + searchfor + " at " + i + ", " + j);
27         }
28         else {
29             System.out.println(searchfor + " not in the array");
30         }
31     }
32 }
```



BreakWithLabelDemo.java

Java Control Flow Statements – Jump/Branching Statements - continue

- There are both **un-labelled continue** and **labelled continue** statements

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/branch.html>

```
1
2 // This program illustrates the usage of the un-labelled continue statement.
3 // Counts the number of occurrences of the latter 'p' in a
4 // given sentence.
5
6 class ContinueDemo_Unlabelled {
7     public static void main(String[] args) {
8
9         String searchMe = "peter piper picked a " + "peck of pickled peppers";
10        int max = searchMe.length();
11        int numPs = 0;
12
13        for (int i = 0; i < max; i++) {
14            // interested only in p's
15            if (searchMe.charAt(i) != 'p')
16                continue;
17
18            // process p's
19            numPs++;
20        }
21        System.out.println("Found " + numPs + " p's in the string.");
22    }
23 }
```

Here is the output of this program:

Found 9 p's in the string.

Java Control Flow Statements – Jump/Branching Statements - continue

Labelled continue statement.

- A labeled continue statement skips the current iteration of an outer loop marked with the given label.

In **Labelled Continue Statement**, we give a *label/name* to a loop.

When this continue statement is encountered with the label/name of the loop, it skips the execution any statement within the loop for the current iteration and *continues* with the next iteration and condition checking in the *labelled loop*.

